

Les Algorithmes Gloutons

NSI - Algorithmique

1. Un problème de taille...

Imaginez que vous êtes un livreur. Vous devez visiter **10 villes** différentes et revenir à votre point de départ.

? Question pour vous : À votre avis, combien de trajets possibles existe-t-il ?

Le problème du Voyageur de Commerce

Pour 10 villes, il y a $9! = 362\,880$ trajets possibles !

Pour 20 villes, c'est de l'ordre de 10^{17} trajets !

Même le plus puissant de nos ordinateurs mettrait des années à tous les tester un par un pour trouver le chemin le plus court (on appelle ça la force brute).

? Et si on devait trouver une solution "acceptable" très vite, comment feriez-vous "à l'instinct" ?

La stratégie de l'instinct : l'approche Gloutonne

Au lieu de calculer TOUS les trajets, on fait le choix qui paraît **le meilleur sur le moment**.

☞ *Règle possible : "Depuis la ville où je suis, je vais à la ville non visitée la plus proche."*

C'est simple, c'est rapide... C'est un **algorithme glouton** !

2. Qu'est-ce qu'un Algorithme Glouton ?

Un algorithme est dit **glouton** s'il construit une solution étape par étape.

À chaque étape, il fait le choix qui semble le meilleur **localement**, avec l'espoir que ces choix optimaux locaux mèneront à une solution optimale **globale**.

Conditions d'utilisation :

- Le problème peut être découpé en une suite de choix.
- Une fois un choix fait, on ne revient **jamais** en arrière ! (Pas de Ctrl+Z)

3. Le problème du rendu de monnaie

Vous êtes caissier. Vous devez rendre 14€ à un client avec le moins de pièces et billets possible.

Votre caisse contient des pièces de 1€, 2€ et des billets de 5€, 10€.

? Comment procédez-vous ? Quelle est votre stratégie "humaine" ?

La logique du rendu de monnaie

La stratégie gloutonne est celle que nous utilisons tous les jours :

On rend toujours la pièce ou le billet de la plus grande valeur possible sans dépasser la somme à rendre !

Pour rendre 14€ :

1. Plus grand billet ≤ 14 ? 🖱️ 10€ (reste 4€)
2. Plus grand billet/pièce ≤ 4 ? 🖱️ 2€ (reste 2€)
3. Plus grand billet/pièce ≤ 2 ? 🖱️ 2€ (reste 0€)

Total : 1 billet de 10€, 2 pièces de 2€. (3 éléments).

💻 **À vous de jouer :** Vous devrez implémenter cet algorithme en Python lors du prochain TP !

4. Le problème du sac à dos (Knapsack Problem)

Vous êtes un aventurier dans un donjon. Votre sac à dos peut contenir au maximum 10 kg.

Vous trouvez 3 objets :

- Un lingot d'or : Poids = 8 kg, Valeur = 8000 €
- Un joyau A : Poids = 5 kg, Valeur = 5000 €
- Un joyau B : Poids = 5 kg, Valeur = 5000 €

? Quels objets prenez-vous pour être le plus riche possible sans craquer le sac ?

Sac à dos : Quelle stratégie gloutonne ?

On peut imaginer plusieurs stratégies gloutonnes. Par exemple :

- **Le plus cher d'abord ?** On prend l'or (8kg, 8000€). Reste 2kg. On ne peut plus rien prendre. Total = 8000€.
- **Le plus léger d'abord ?** On prend Joyau A et B. (10kg, 10000€).

Ici, la stratégie gloutonne "prendre le plus cher" n'a pas donné la meilleure solution possible !

 **Mission TP :** Vous allez créer un programme pour simuler ces différentes stratégies sur des listes d'objets !

5. Les limites des Algorithmes Gloutons

Vous venez de le voir avec le sac à dos : un choix localement optimal ne donne pas toujours la solution globalement optimale !

Un autre exemple avec la monnaie :

Imaginez un système monétaire inventé avec des pièces de 1, 3, 4.

Il faut rendre 6.

- L'algo glouton donne : 4, puis 1, puis 1 (3 pièces).
- La solution optimale est en fait : 3 et 3 (2 pièces seulement !).

Bilan

✓ Avantages :

- Simples à comprendre et à concevoir.
- Très rapides en temps d'exécution.
- Donnent souvent une "bonne" solution (une approximation satisfaisante), même si elle n'est pas parfaite.

✗ Inconvénients :

- Ne garantissent pas toujours de trouver LA meilleure solution absolue.

? Avez-vous des questions avant de passer à la pratique sur Python ?